# Spring 2026 Analytics II Team Project Final Report

**Group 1**

Joshua Boyles
Dave Julianto
Hayden Burke
Daniel Veiner
Sebastian Hajdu

**Professor:**

Dr. Kaixun Hua

**Date:**

May 1, 2026

## Abstract

Manufacturing defect detection is critical for maintaining product quality, reliability, and safety in industrial systems. In manufacturing, undetected defective components can lead to equipment failures, increased maintenance costs, scrap, rework, and potential safety risks. This project applies machine learning techniques to predict defective parts using historical manufacturing test data.

The dataset contains 229,999 observations, including 850 duplicate records, 0 missing values, and 30 numerical predictor variables representing sensor readings or process measurements. The target variable is a binary defect label, where Class 1 represents defective parts. Only 458 samples are defective, giving a defect rate of 0.1991%. This creates a severe imbalanced classification problem, where traditional metrics such as accuracy can be misleading because a model may achieve high accuracy while failing to identify rare defects.

To address this challenge, this project first compares several machine learning models. Because the defective class is extremely rare, each model is tested with different imbalance-handling strategies, including oversampling, undersampling, SMOTE, and class-weighted learning. Different data splits are also compared to check whether performance remains stable. Finally, models are ranked using a custom FN & FP Score based on false negative and false positive rates, since overall accuracy can appear high even when rare defects are missed.


## 1 Introduction

Manufacturing defect detection plays an important role in improving product quality, reducing waste, and preventing costly failures. In manufacturing, defective components can contribute to equipment damage, increased maintenance, production delays, customer dissatisfaction, and potential safety risks if they are not detected early. As manufacturing systems collect more process and test data, machine learning provides an opportunity to identify defect patterns that may not be obvious through manual inspection alone.

This project uses historical manufacturing data to predict whether a manufactured part is defective. Of the full dataset, only 0.1991% of samples are labeled as defective. This makes the problem a severe imbalanced classification task.

The main challenge is that the defective class is extremely rare. This project first compares several machine learning models on this unbalanced data. Because the defective class is extremely rare, each model is tested with different imbalance-handling strategies. Different data splits are also compared to check whether performance remains stable. Finally, models are ranked using a custom FN & FP Score based on false negative and false positive rates, since overall accuracy can appear high even when rare defects are missed.

The objectives of this project are to:

1. Develop a predictive model for classifying defective parts.
2. Evaluate model performance under severe class imbalance using the custom FN & FP Score.
3. Identify key process variables associated with defective production.
4. Provide insights that can support manufacturing quality control and process improvement.

Together, these objectives outline a framework for using historical manufacturing data to improve defect detection.

## 2 Related Work

Machine learning has increasingly been applied to manufacturing systems for predictive quality monitoring and defect detection. Prior research shows that industrial process and test data can be used to learn relationships between operating conditions and product quality outcomes.

Zahrani et al. apply machine learning models to predict product quality and groove geometry in a circular laser-grooving manufacturing process. Their results show that data-driven models can capture nonlinear relationships between process parameters and final part characteristics, supporting predictive monitoring rather than purely reactive inspection [1].

More recent studies extend this direction using stronger ensemble methods for rare-defect prediction in industrial environments. Geng et al. propose a framework for predicting internal cracks in continuous steel casting using CatBoost combined with synthetic data generation, demonstrating improved detection of infrequent defect patterns in a real manufacturing context [2].

A central challenge in manufacturing defect prediction is severe class imbalance. Defective products may represent only a small fraction of observations, causing standard training and evaluation to favor the majority class and report high accuracy while missing rare failures. Kafunah et al. examine this issue in fault detection and evaluate imbalance-aware strategies such as oversampling and reweighting to improve minority-class detection, highlighting the importance of focusing on defect-class metrics such as precision, recall, and F1 rather than accuracy alone [3].

Cost-sensitive learning provides a complementary approach by explicitly assigning higher penalties to misclassifying defective samples, reflecting the higher operational risk of false negatives in manufacturing. Liu et al. introduce a cost-sensitive defect detection framework that modifies the training objective using an explicit cost structure to prioritize costly minority-class errors, and they report improved defect identification under imbalance [4].

Building on these findings, our project systematically compares interpretable and ensemble models, including logistic regression, decision trees, random forests, and boosting under multiple imbalance-mitigation strategies. We emphasize cost-sensitive random forests and minority-class performance metrics to evaluate model effectiveness for defect detection in a highly imbalanced manufacturing dataset.

# 3 Model and Method

## 3.1 Class Imbalance Strategies

The dataset comprises 229,999 test records with only 458 defective samples (0.2%). To address the imbalance, we evaluate four dataset manipulation techniques: Random Oversampling (duplicating minority samples), Random Undersampling (removing majority samples), SMOTE (synthetic sample generation), and Class-Weighted training (adjusting learning objectives to penalize minority-class errors). Each data sampling method was tested on every considered model type.

## 3.2 Considered Model Types

### Logistic Regression

Logistic Regression provides high interpretability, modeling defect probability as a smooth function of sensor readings so engineers can quantify feature impact:

$$P(defective|x) = \sigma(\beta_0 + \beta^\top x) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x)}} \tag{1}$$

### Decision Trees

Decision Trees sort data through binary splits based on sensor thresholds. We use Gini Impurity, as it is numerically stable under extreme class imbalance:

$$Gini(S) = \sum_{k \in \{0,1\}} p_k(1 - p_k) \tag{2}$$

Splits are evaluated by their reduction in impurity:

$$\Delta = Gini(S) - \left( \frac{|S_L|}{|S|} Gini(S_L) + \frac{|S_R|}{|S|} Gini(S_R) \right) \tag{3}$$

Based on 5-Fold Cross Validation, the optimal maximum depth was set to 5 to prevent overfitting.

**Random Forest**

Random Forest aggregates multiple trees trained on bootstrap samples and random feature subsets, yielding a majority vote prediction:

$$\hat{y} = \text{mode}\{\hat{y}^{(1)}, \hat{y}^{(2)}, \ldots, \hat{y}^{(N)}\} \tag{4}$$

We evaluate both Balanced Random Forest and Cost-Sensitive Random Forest to address the severe class imbalance in the dataset. Balanced Random Forest uses internal undersampling for each tree, meaning each bootstrap sample is adjusted so that the defective class is better represented during training. Cost-Sensitive Random Forest keeps the original training distribution but applies class-weighted impurity, giving higher penalty to mistakes on defective samples. Comparing both approaches allows us to test whether defect detection improves more from changing the sample distribution or from changing the cost of minority-class errors.

**Boosting (AdaBoost, XGBoost, LightGBM, CatBoost)**

Boosting builds an ensemble by repeatedly reweighting samples to focus on previous mistakes. For our baseline, AdaBoost, the error is:

$$\epsilon_i = \sum_j w_j^i \delta(C_i(x_j) \neq y_j) \tag{5}$$

Classifier importance:

$$\alpha_t = \frac{1}{2} \ln \left( \frac{1 - \epsilon_t}{\epsilon_t} \right) \tag{6}$$

Sample weights update:

$$w_j^{i+1} = \frac{w_j^i}{Z_i} \times \begin{cases} e^{-\alpha_i}, & C_i(x_j) = y_j \\ e^{\alpha_i}, & C_i(x_j) \neq y_j \end{cases} \tag{7}$$

We extend our analysis to advanced gradient boosting frameworks (XGBoost, LightGBM, and CatBoost) to gauge performance gains and processing efficiency against the AdaBoost baseline.

### 3.3 Data Split Comparison

Testing model generalization across 90/10, 80/20, and 70/30 train/test split ratios.

### 3.4 Hyperparameter Tuning

A 5-Fold Cross Validation method was done to tune the hyperparameters for each model type on the baseline 80/20 split:

Table 1: Boosting Models Hyperparameters: Default vs Tuned

| Model | Parameter | Default | Tuned |
|---|---|---|---|
| Decision Tree | max_depth | 5 | 5 |
| AdaBoost | n_estimators | 50 | 200 |
| | learning_rate | 1.0 | 1.5 |
| XGBoost | n_estimators | 100 | 300 |
| | learning_rate | 0.3 | 0.05 |
| LightGBM | n_estimators | 100 | 300 |
| CatBoost | iterations | 1000 | 300 |

# 4 Experimental Design

## 4.1 Model Selection Criteria

We evaluate simple models alongside complex ensembles to consider the trade-off between interpretability and predictive performance. Our experimental phase primarily evaluates 120 unique scenarios via our custom "FN & FP Score" System, as well as secondary evaluation with other relevant metrics in the case that predictive ability and interpretability do not significantly emphasize a best choice model.

Conceptual model transparency and interpretability are considered for final model suggestion/recommendation.

## 4.2 "FN & FP Score" System for Evaluating Model Prediction Ability

In many classification tasks, reducing false negatives (FN) is often more critical than reducing false positives (FP). To systematically evaluate models with this consideration, we propose a weighted scoring system that incorporates both the false negative rate (FNR) and false positive rate (FPR).

**Weighted Score Definition.** We define the **"FN & FP Score"** for a model as a weighted sum of the FN Rate and FP Rate:

$$\text{FN \& FP Score} = (\text{FN weighting}) \times (\text{FN Rate}) + (\text{FP weighting}) \times (\text{FP Rate}) \tag{8}$$

where FN weighting and FP weighting are non-negative weights reflecting the relative importance of reducing false negatives versus false positives.

**Auxiliary Equation.** Assuming the weights are complementary, we can express the FP weighting in terms of the FN weighting:

$$\text{FP weighting} = 1 - \text{FN weighting} \tag{9}$$

**Equivalent Equation.** Substituting the complementary weighting into the score formula yields:

$$\text{FN \& FP Score} = (\text{FN weighting}) \times (\text{FN Rate}) + (1 - \text{FN weighting}) \times (\text{FP Rate}) \tag{10}$$

This equivalent equation shows that the score is a function of three components: FN weighting, FNR, and FPR.

**Model Ranking Using the "FN & FP Score".** The "FN & FP" Score allows ranking models according to their performance while prioritizing false negatives. For a given FN weighting, the **lowest score** indicates the **best model** at that weighting. By varying FN weighting (e.g., 50%, 60%, ..., 100%), we can evaluate model performance under different prioritization schemes.

This approach provides a flexible and interpretable framework for model selection, emphasizing the trade-off between FN and FP according to domain-specific requirements.

## 4.3 Other Relevant Metrics

Due to the extreme class imbalance, accuracy and ROC AUC can be misleading metrics [5]. Instead, recall, precision, Area Under the Precision-Recall Curve (PR-AUC), and the F1-minority score can be used as secondary metrics to evaluate our models. Summary tables of these metrics are embedded within our attached Juptyer Notebook. Theses metrics serve as secondary evaluative criteria of our models in the case that predictive ability and interpretability do not significantly emphasize a best choice model.

# 5 Results

## 5.1 Comparing Predicitve Ability

## 5.2 Comparing Interpretability

# 6 Conclusion and Future Work

## 6.1 Our Recommended Model

## 6.2 Limitations of Our Study

## 6.3 Potential Future Improvements

## References


[1] E. G. Zahrani, F. Hojati, A. Daneshi, B. Azarhoushang, and J. Wilde. Application of machine learning to predict the product quality and geometry in circular laser grooving process. *Procedia CIRP*, 94:474–480, 2020.

[2] M. Geng, H. Ma, S. Liu, Z. Zhou, L. Xing, Y. Ai, and W. Zhang. A data-driven approach for internal crack prediction in continuous casting of HSLA steels using CTGAN and CatBoost. *Materials*, 18:3599, 2025.

[3] J. Kafunah, M. I. Ali, and J. G. Breslin. Handling imbalanced datasets for robust deep neural network-based fault detection in manufacturing systems. *Applied Sciences*, 11:9783, 2021.

[4] B. Liu, F. Gao, and Y. Li. Cost-sensitive YOLOv5 for detecting surface defects of industrial products. *Sensors*, 23:2610, 2023.

[5] M. Imani, M. Joudaki, A. Bagheri, and H. R. Arabnia. Why ROC-AUC is misleading for highly imbalanced data: In-depth evaluation of MCC, F2-Score, H-Measure, and AUC-based metrics across diverse classifiers. *Technologies*, 14(1):54, 2026.

[6] ScienceDirect. Minority class. *ScienceDirect Topics*. Retrieved March 5, 2025.

# Appendix

## A. Detailed Scoring Results

Table 2: "FN & FP Scores" and Error Rates for Various Scenarios

| FN Weight | Rank | Scenario | "FN & FP Score" | FNR | FPR |
|---|---|---|---|---|---|
| 0.5 | 1 | 90/10 \| Random Undersampling \| CatBoost | 0.034292137 | 0.045454545 | 0.023129728 |
| 0.5 | 2 | 90/10 \| Random Oversampling \| Logistic Regression | 0.044256621 | 0.068181818 | 0.020331424 |
| 0.5 | 3 | 90/10 \| Class-Weighted \| Logistic Regression | 0.044322207 | 0.068181818 | 0.020462595 |
| 0.5 | 4 | 80/20 \| SMOTE \| Logistic Regression | 0.045513672 | 0.068181818 | 0.022845525 |
| 0.5 | 5 | 90/10 \| SMOTE \| Logistic Regression | 0.045524603 | 0.068181818 | 0.022867387 |
| 0.5 | 6 | 90/10 \| Random Undersampling \| AdaBoost | 0.046884502 | 0.045454545 | 0.048314459 |
| 0.5 | 7 | 90/10 \| Random Undersampling \| LightGBM | 0.047186095 | 0.068181818 | 0.026190372 |
| 0.5 | 8 | 90/10 \| Random Undersampling \| XGBoost | 0.047448436 | 0.068181818 | 0.026715054 |
| 0.5 | 9 | 80/20 \| Random Undersampling \| Logistic Regression | 0.047538119 | 0.056818182 | 0.038258056 |
| 0.5 | 10 | 90/10 \| Random Undersampling \| Logistic Regression | 0.047907533 | 0.068181818 | 0.027633247 |
| 0.6 | 1 | 90/10 \| Random Undersampling \| CatBoost | 0.036524618 | 0.045454545 | 0.023129728 |
| 0.6 | 2 | 90/10 \| Random Undersampling \| AdaBoost | 0.046598511 | 0.045454545 | 0.048314459 |
| 0.6 | 3 | 90/10 \| Random Oversampling \| Logistic Regression | 0.04904166 | 0.068181818 | 0.020331424 |
| 0.6 | 4 | 90/10 \| Class-Weighted \| Logistic Regression | 0.049094129 | 0.068181818 | 0.020462595 |
| 0.6 | 5 | 80/20 \| Random Undersampling \| Logistic Regression | 0.049394132 | 0.056818182 | 0.038258056 |
| 0.6 | 6 | 80/20 \| SMOTE \| Logistic Regression | 0.050047301 | 0.068181818 | 0.022845525 |
| 0.6 | 7 | 90/10 \| SMOTE \| Logistic Regression | 0.050056046 | 0.068181818 | 0.022867387 |
| 0.6 | 8 | 90/10 \| Random Undersampling \| LightGBM | 0.05138524 | 0.068181818 | 0.026190372 |
| 0.6 | 9 | 90/10 \| Random Undersampling \| XGBoost | 0.051595112 | 0.068181818 | 0.026715054 |
| 0.6 | 10 | 90/10 \| Random Undersampling \| Logistic Regression | 0.05196239 | 0.068181818 | 0.027633247 |
| 0.7 | 1 | 90/10 \| Random Undersampling \| CatBoost | 0.0387571 | 0.045454545 | 0.023129728 |
| 0.7 | 2 | 90/10 \| Random Undersampling \| AdaBoost | 0.046312519 | 0.045454545 | 0.048314459 |
| 0.7 | 3 | 80/20 \| Random Undersampling \| Logistic Regression | 0.051250144 | 0.056818182 | 0.038258056 |
| 0.7 | 4 | 90/10 \| Random Oversampling \| Logistic Regression | 0.0538267 | 0.068181818 | 0.020331424 |
| 0.7 | 5 | 90/10 \| Class-Weighted \| Logistic Regression | 0.053866051 | 0.068181818 | 0.020462595 |

| FN Weight | Rank | Scenario | "FN & FP Score" | FNR | FPR |
|---|---|---|---|---|---|
| 0.7 | 6 | 80/20 \| SMOTE \| Logistic Regression | 0.05458093 | 0.068181818 | 0.022845525 |
| 0.7 | 7 | 90/10 \| SMOTE \| Logistic Regression | 0.054587489 | 0.068181818 | 0.022867387 |
| 0.7 | 8 | 90/10 \| Random Undersampling \| LightGBM | 0.055584384 | 0.068181818 | 0.026190372 |
| 0.7 | 9 | 90/10 \| Random Undersampling \| XGBoost | 0.055741789 | 0.068181818 | 0.026715054 |
| 0.7 | 10 | 90/10 \| Random Undersampling \| Logistic Regression | 0.056017247 | 0.068181818 | 0.027633247 |
| 0.8 | 1 | 90/10 \| Random Undersampling \| CatBoost | 0.040989582 | 0.045454545 | 0.023129728 |
| 0.8 | 2 | 90/10 \| Random Undersampling \| AdaBoost | 0.046026528 | 0.045454545 | 0.048314459 |
| 0.8 | 3 | 80/20 \| Random Undersampling \| Logistic Regression | 0.053106157 | 0.056818182 | 0.038258056 |
| 0.8 | 4 | 90/10 \| Random Oversampling \| Logistic Regression | 0.058611739 | 0.068181818 | 0.020331424 |
| 0.8 | 5 | 90/10 \| Class-Weighted \| Logistic Regression | 0.058637973 | 0.068181818 | 0.020462595 |
| 0.8 | 6 | 80/20 \| SMOTE \| Logistic Regression | 0.059114559 | 0.068181818 | 0.022845525 |
| 0.8 | 7 | 90/10 \| SMOTE \| Logistic Regression | 0.059118932 | 0.068181818 | 0.022867387 |
| 0.8 | 8 | 90/10 \| Random Undersampling \| LightGBM | 0.059783529 | 0.068181818 | 0.026190372 |
| 0.8 | 9 | 90/10 \| Random Undersampling \| XGBoost | 0.059888465 | 0.068181818 | 0.026715054 |
| 0.8 | 10 | 90/10 \| Random Undersampling \| Logistic Regression | 0.060072104 | 0.068181818 | 0.027633247 |
| 0.9 | 1 | 90/10 \| Random Undersampling \| CatBoost | 0.043222063 | 0.045454545 | 0.023129728 |
| 0.9 | 2 | 90/10 \| Random Undersampling \| AdaBoost | 0.045740536 | 0.045454545 | 0.048314459 |
| 0.9 | 3 | 80/20 \| Random Undersampling \| Logistic Regression | 0.054962169 | 0.056818182 | 0.038258056 |
| 0.9 | 4 | 90/10 \| Random Oversampling \| Logistic Regression | 0.063396779 | 0.068181818 | 0.020331424 |
| 0.9 | 5 | 90/10 \| Class-Weighted \| Logistic Regression | 0.063409896 | 0.068181818 | 0.020462595 |
| 0.9 | 6 | 80/20 \| SMOTE \| Logistic Regression | 0.063648189 | 0.068181818 | 0.022845525 |
| 0.9 | 7 | 90/10 \| SMOTE \| Logistic Regression | 0.063650375 | 0.068181818 | 0.022867387 |
| 0.9 | 8 | 90/10 \| Random Undersampling \| LightGBM | 0.063982673 | 0.068181818 | 0.026190372 |
| 0.9 | 9 | 90/10 \| Random Undersampling \| XGBoost | 0.064035142 | 0.068181818 | 0.026715054 |
| 0.9 | 10 | 90/10 \| Random Undersampling \| Logistic Regression | 0.064126961 | 0.068181818 | 0.027633247 |
| 1 | 1 | 90/10 \| Random Undersampling \| CatBoost | 0.045454545 | 0.045454545 | 0.023129728 |
| 1 | 2 | 90/10 \| Random Undersampling \| AdaBoost | 0.045454545 | 0.045454545 | 0.048314459 |
| 1 | 3 | 80/20 \| Random Undersampling \| Logistic Regression | 0.056818182 | 0.056818182 | 0.038258056 |
| 1 | 4 | 80/20 \| Random Undersampling \| CatBoost | 0.068181818 | 0.068181818 | 0.027720694 |

| FN Weight | Rank | Scenario | "FN & FP Score" | FNR | FPR |
|---|---|---|---|---|---|
| 1 | 5 | 80/20 \| Random Undersampling \| LightGBM | 0.068181818 | 0.068181818 | 0.030693892 |
| 1 | 6 | 90/10 \| Random Oversampling \| Logistic Regression | 0.068181818 | 0.068181818 | 0.020331424 |
| 1 | 7 | 90/10 \| Random Undersampling \| LightGBM | 0.068181818 | 0.068181818 | 0.026190372 |
| 1 | 8 | 90/10 \| SMOTE \| Logistic Regression | 0.068181818 | 0.068181818 | 0.022867387 |
| 1 | 9 | 80/20 \| SMOTE \| Logistic Regression | 0.068181818 | 0.068181818 | 0.022845525 |
| 1 | 10 | 90/10 \| Class-Weighted \| Logistic Regression | 0.068181818 | 0.068181818 | 0.020462595 |

## B. Bar Charts: Top 10 Models Per Weighting

The following bar charts include the top 10 scoring models for each False Negative weight (50%-100%), along with their "FN & FP Scores".

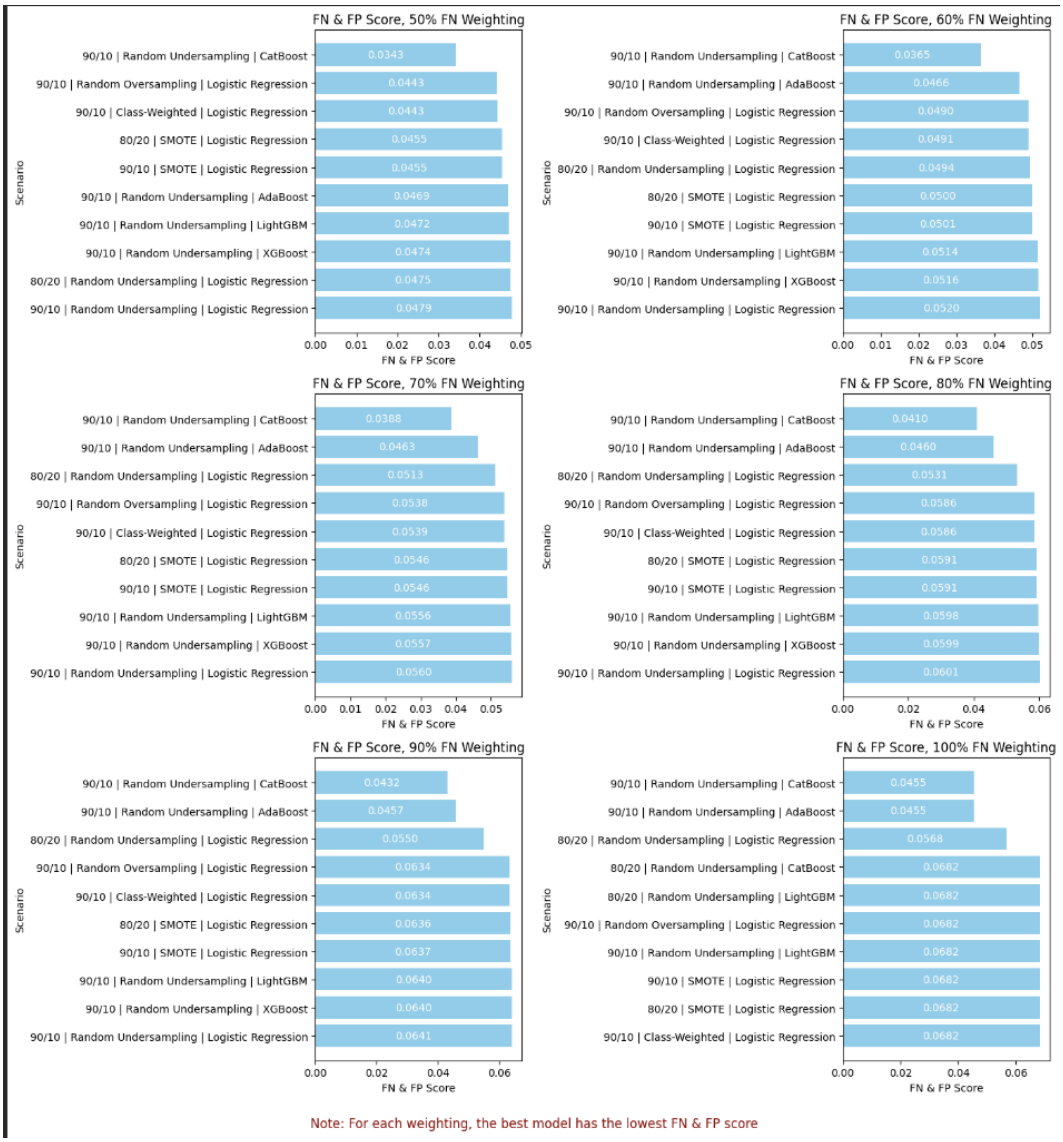


Figure 1: Model ranking results using the custom FN & FP Score.